



Supporting Agile Organizations with a Decision Guidance Query Language

Alexander Brodsky , Nathan E. Egge & X. Sean Wang

To cite this article: Alexander Brodsky , Nathan E. Egge & X. Sean Wang (2012) Supporting Agile Organizations with a Decision Guidance Query Language, Journal of Management Information Systems, 28:4, 39-68

To link to this article: <http://dx.doi.org/10.2753/MIS0742-1222280403>



Published online: 08 Dec 2014.



Submit your article to this journal [↗](#)



Article views: 20



View related articles [↗](#)

Supporting Agile Organizations with a Decision Guidance Query Language

ALEXANDER BRODSKY, NATHAN E. EGGE, AND
X. SEAN WANG

ALEXANDER BRODSKY is director of the Center for Smart Power Grids and an associate professor of Computer Science at George Mason University. He earned his Ph.D. and prior degrees in computer science and mathematics from the Hebrew University of Jerusalem. His research interests and publications are in the areas of decision-guidance and support systems, constraint databases and programming, secure information sharing, and smart power grids. For his research work on constraint databases and programming, Dr. Brodsky received a National Science Foundation (NSF) CAREER Award, NSF Research Initiation Award, and grants from the Office of Naval Research and NASA. Dr. Brodsky served as conference chairman of the Fifth International Conference on Principles and Practice of Constraint Programming and serves as a program committee co-chair of the IEEE International Conference on Data Engineering workshop on Decision Guidance and Support Systems. Prior to joining George Mason University, Dr. Brodsky worked at IBM's T.J. Watson Research Center, at Israel Aircraft Industries, and was an officer in the Computer Division of Communications, Electronics and Computer Corps, Israel Defense Forces.

NATHAN E. EGGE is a Ph.D. candidate at George Mason University working on a dissertation on the decision-guidance query language. He holds dual master's degrees in mathematics and computer science from Virginia Tech. His research interests include decision-guidance and support systems, database management systems, constraint and mathematical programming, distributed computing, and enterprise software architecture. He has over 10 years' experience in the industry, designing and building enterprise applications for Fortune 500/Global 1000 companies.

X. SEAN WANG is dean of the School of Computer Science at Fudan University, Shanghai, China. Prior to joining Fudan University, Dr. Wang was a program officer at the U.S. National Science Foundation, and endowed Dorothean Chair and professor of computer science in the College of Engineering and Mathematics at the University of Vermont. He started his academic career at the Information and Software Engineering Department of George Mason University, after earning his Ph.D. in computer science from the University of Southern California, and B.S. and M.S. in computer science from Fudan University. Dr. Wang's research areas include database management systems, information security and privacy, wireless sensor networks, streaming data processing, time series queries, data mining and data warehousing, and database theory. He is the principal investigator or co-investigator of close to a dozen federally sponsored research projects and is a recipient of the National Science Foundation's Career and Research Initiation Awards. He has published over 100 scientific papers in the general field of database systems.

ABSTRACT: Decision optimization is widely used in many decision guidance and support systems (DGSS) to support business decisions such as procurement, scheduling, and planning. In spite of rapid changes in users' requirements, the implementation of DGSS is typically rigid, expensive, and not easily extensible, which is in stark contrast to the agile implementation of management information systems (MIS) based on the database management systems (DBMS) and SQL technologies. This paper focuses on the Decision Guidance Query Language (DGQL) designed to (re-)use SQL programs for decision optimization with the goals of making DGSS implementation agile and intuitive and leveraging existing investment in SQL-implemented MIS. The paper addresses two related technical issues with DGQL: (1) how to annotate existing queries to precisely express the optimization semantics, and (2) how to translate the annotated queries into equivalent mathematical programming formulations that can be solved efficiently.

KEY WORDS AND PHRASES: agile organizations, decision guidance, decision support, optimization.

BUSINESS MANAGERS FACED WITH GLOBAL COMPETITION and rapidly changing market conditions are increasingly turning to decision guidance and support systems (DGSS) to provide actionable recommendations on a variety of business decisions. Examples include manufacturing processes, sourcing, planning and scheduling, transportation and delivery, and inventory control. While rapid change in users' requirements is typical for DGSS, the development of DGSS today is a rigid, time-consuming, and expensive process. Furthermore, existing DGSS typically are difficult to modify or extend. This paper makes a step toward "agile" development and extensibility of DGSS using the Decision Guidance Query Language (DGQL).

DGSS can be viewed as a class of information systems (IS) that provide users not only business data to support decision making but also actionable business recommendations. As observed in Hong et al. [19], to address the gap between rapid changes in users' requirements and the high cost of development software [3, 22], so-called "agile" software development methods, such as Extreme Programming, have evolved. Agile methods, as described in Hong et al., are a "collection of principles and techniques that emphasize early and continuous delivery of software" [19, p. 236] in response to rapid changes in users' requirements [9, 28]. However, agile development of DGSS has not been addressed due to unique characteristics and challenges of DGSS.

The challenge in building agile DGSS has to do with the fact that its state-of-the-art implementation today involves mathematical programming (MP) and constraint programming (CP). MP and CP are used for decision optimization, that is, finding values for control variables that maximize or minimize an objective within given constraints. More specifically, the development of DGSS usually involves (1) analysis and formal modeling of an optimization problem by an operations research expert, (2) implementation of a model using a modeling language such as AMPL (A Mathematical Programming Language) and MP/CP solvers, and further refinement of algorithms by MP/CP experts, and (3) development of the application IS and its integration with the

optimization module by software developers. Software and IS application developers normally do not possess the expertise for the DGSS development cycle. Furthermore, a small change in application requirements often leads to a complete redesign of the formal optimization model and its implementation. Clearly, these challenges hinder agile development of DGSS.

In contrast, management information systems (MIS) software developers usually are familiar with database management systems (DBMS) and the SQL query language. Using DBMS and SQL allows a significantly more agile development because the developers can focus on declarative query and business logic while leaving the system details (such as efficient query evaluation and transaction management, as well as failure and recovery mechanisms) to DBMS. While nearly all IS software developers are familiar with DBMS and SQL, they typically do not have the mathematical expertise necessary for MP and CP. Furthermore, DGSS are usually domain specific; require significant expertise to develop, deploy, and maintain; and often lack the return on investment (ROI) necessary to justify investment [17]. In addition, much investment has already been spent on database application development and infrastructure. Clearly, it is desirable to leverage this investment when building DGSS.

In this paper, we describe a DGQL framework. With DGQL, we provide a step toward making decision optimization easier and more agile, especially in applications where database technology has been heavily used. Briefly, DGQL provides a query-like abstraction for expressing decision optimization problems so that database programmers can use it without prior experience in MP/CP, and more importantly, so that they can reuse SQL programs already built into existing MIS.

A key observation that motivates the DGQL development is that database languages such as SQL are intuitive to use for MIS, and that decision optimization in principle is an “inverse” of much of the SQL functionality in place. In addition, code used in SQL programs often contains business logic that is needed in the decision optimization tasks. Based on this observation, a decision optimization problem in DGQL is written as a “regular” SQL program, that is, a sequence of relational views and accompanying integrity constraints, together with some annotation of which database table column needs to be decided by the system (i.e., variables) and toward what goal (i.e., optimization objective). Here, existing SQL programs in MIS can be directly used. Essentially, DGQL allows users to write an optimization problem as if writing an SQL program in a forward manner.

The technical challenge in the DGQL approach, however, is how to execute the “inverse” decision optimization based on a “forwardly” expressed code. This is done by encoding the DGQL queries as an MP formulation, solving the MP problem, and then deriving the solution to the DGQL optimization problem. A technical question with two interrelated parts arises: Is it possible to (1) encode a DGQL program as an MP formulation such that (2) the solution can be found efficiently? In this paper, we answer this question affirmatively. As such, we believe that DGQL has the potential to achieve both the agility of DGSS development and the efficiency of optimization solutions. In terms of efficiency, the overall performance of intuitive DGQL queries compares squarely with expert-generated MP problems, as we show in the seventh section.

A common problem faced with the proposal of any new framework is acceptance and usage by the greater research community. By design, DGQL extends the most broadly used database query language, SQL, and adds only a minimal annotation that allows translation of SQL queries into optimization problems. We believe that this approach has a good chance of being broadly used exactly because it does not require database programmers to learn a new language. The DGQL research prototype has been implemented and integrated with the PostgreSQL database server so that it may be invoked directly from any application or interface that allows ordinary SQL statement execution, including the Postgres development environment pgAdmin.

In summary, the contributions of the paper are as follows:

- we introduce the DGQL database language for decision optimization and define its formal syntax and semantics,
- we provide a reduction method to automatically transform DGQL programs into formal MP models and prove its correctness, and
- we report experiments to show the efficiency of a DGQL implementation against expert-generated models.

The rest of this paper is organized as follows. In the next section we give a motivating, running example. In the third section we briefly survey state-of-the-art decision optimization methods and point out a few related works. In the fourth section we use the running example to present DGQL informally in an SQL-like syntax before we present the formal syntax and semantics of DGQL in the fifth section. An implementation of DGQL via translation to MP is given in the sixth section. We present our experimental results in the seventh section, and conclude with the eighth section. Finally, Appendix A adds more detail on the reduction procedure and Appendix B provides a formal proof of correctness.

A Running Example

TO MAKE OUR DISCUSSION CONCRETE, consider first a simple example of a reporting MIS to support a procurement officer who manages the purchase and delivery of various items to delivery locations. Let us assume that a database is set up with the following tables:

```
demand(loc, item, requested):
    a tuple indicates that for a delivery location (loc), the quantity requested of
    item is needed.

supply(vendor, item, price_per_unit, available):
    a tuple indicates that a vendor can provide at most available quantity of
    item at the price_per_unit.

orders(vendor, item, loc, quantity):
    a tuple indicates the order of item in quantity from vendor for location
    loc.
```

With the above tables, the MIS may use the following query to provide (total) cost information:

```
CREATE VIEW total_cost AS
SELECT SUM(s.price_per_unit * o.quantity) AS total
FROM orders o INNER JOIN supplier s
ON o.vendor = s.vendor AND o.item = s.item;
```

The above assumes that the item orders are decided elsewhere, and that the database is simply to record these orders.

Now consider the situation that the procurement officer needs to decide these orders, with the objective of minimizing the cost. Note that we now add a DGSS functionality that goes beyond reporting. This is a typical “inverse” problem of the above SQL function. More specifically, we need to assume that the `orders` table needs to be filled such that all the demands are satisfied while the total cost is minimized. This is a nontrivial problem, especially when there are constraints such as limiting the number of vendors to supply a particular location.

State-of-the-art solutions will need to model the situation in a separate decision optimization system to make the order decisions. Our approach is to reuse the above SQL program, plus some auxiliary SQL statements and SQL-like annotations (e.g., for adding the constraints), to automatically generate MP models that will generate the optimal orders. The resulting annotated SQL query is called a DGQL query. We deliberately make the above query rather simple for easy presentation. One may imagine that more tables may be involved and some investment has already been made to debug and test the queries to make the business logic correctly reflected. This ability to leverage existing investment in database design through annotation is a distinct advantage over modeling languages such as AMPL, GAMS (General Algebraic Modeling System), and OPL (Optimization Programming Language). This is because DGQL allows a clear separation between modeling the business objects and processes, and indicating which parameters are to be used for decision making. One can imagine a multistage decision optimization system where the output of one decision problem is used as the input to another. By allowing for heavy query reuse, new optimization problems can be created by a few annotations with little advanced planning.

Furthermore, additional information may be added to the system at different times, such as shipping options, packaging options, and so forth. Consider shipping options. As multiple shipping companies may be available with different pricing structures, it is useful to optimize the total shipping cost with the best shipping option. We may have two additional tables: `shipping_options` and `shipping_orders`. An additional query can be added to provide total shipping cost. (Care needs to be taken to make the item orders and `shipping_orders` consistent, perhaps by using integrity constraints in the database.) An interesting point here is that relational database systems allow modularized development, which increases the productivity of the programmers and makes the system development more intuitive.

With the additional needed information, the procurement officer may need to also make shipping (and other) decisions so that the total cost of items and shipping is

minimized. Other requirements may come in as well, such as the time of delivery requirements. Again we can “invert” the SQL function to make decision optimizations by reusing the SQL code already developed. If shipping option is added to the item orders, two tables, namely, `item_orders` and `shipping_orders`, need to be filled to make the total cost optimal. The DGQL query will be modular in exactly the same way that SQL queries are modular.

Related Work

AS INDICATED IN HONG ET AL. [19], the existing research on agile IS focused on agile methods for both software developers (e.g., [1, 7, 26]) and project managers (e.g., [15, 18, 23, 29]). Agile IS were also studied from the perspective of user acceptance [19]. A book by Desouza [11] deals with different aspects of agile IS, including conceptualization, construction, and management. However, the problem of making DGSS agile has not been addressed. Note that the work on agile IS is not sufficient to make DGSS development agile because of unique challenges in developing DGSS.

The reason for the uniqueness is that the development of DGSS, even the simple example in the previous section, requires both database management and operations research (OR) techniques and tools. Using OR tools such as MP and CP poses significant challenges. Among them is the requirement of full knowledge of the search space and the objective for the task at hand for efficient system operation. The OR modeling abstraction (e.g., AMPL [13] and GAMS [5]) typically requires OR expertise, which makes the development by information technology (IT) professionals challenging. In contrast, DBMS tools are more intuitive and have been adopted by many application domains. However, DBMS query languages are not designed for decision optimization as they cannot express decision optimization problems, notably over continuous variables. Indeed, in the continuous variable case, there are potentially infinite possibilities to choose from that cannot be expressed as a regular database query program. For the discrete case, when potential choices are from a large space, populating tables with all possible choices and then performing a rank query can be quite inefficient. Although database query languages, such as SQL, can handle some limited discrete optimization computations (e.g., find a tuple that has a minimal value over a finite set of discrete choices [20] even in the cases of expressible rank queries), evaluation algorithms have not typically taken advantage of MP and CP search strategies to achieve potential efficiency and flexible optimization goals. More important, the optimization program will look very different from the database program query, increasing the complexity of system management.

Specialized optimization tools (e.g., for optimizing price-revenue, transportation, sourcing, production planning) have been developed to be ready for end users, or for integration with other systems (e.g., enterprise resource planning or enterprise resource management), so that operations research expertise would not be necessary. However, this approach is not extensible, and does not support general DGSS development.

There has been extensive work on integrating data manipulation and mathematical programming modeling languages (e.g., [2, 4, 8, 12, 24]). The work of Dolk [12] strongly advocates the need to unify models such as optimization, econometric, simu-

lation, and data models. Dolk's article gives an excellent introduction and overview to model integration and integrated modeling environments, including the integration of schema, process, models and data, models and solvers, modeling paradigms, environments, and finally, modeling system interfaces. Bhargava et al. [4] present a methodology and provide systematic means for integrating data and mathematical modeling languages. In their work, Mitra et al. [24] extend the syntax of Mathematical Programming Language (MPL) so that the index sets used in the constraint formulation are extracted from a relational database in a flexible fashion. Choobineh [8] makes a tight integration of SQL and a mathematical programming language by creating an SQL-like syntax to define constraints and extract data from the database. Atamtürk et al. [2] take a similar approach and propose using SQL to specify "blocks" that represent matrices for linear programming models. The above work on integration is significant because it makes implementation of systems easier due to tighter integration of MP modeling with data extraction and manipulation.

However, in all these works, whether data manipulation and MP functionalities are integrated in a looser or tighter fashion, users must still work with two conceptually different paradigms: (1) data manipulation and (2) definition of MP problems. Thus, the user of those systems must still possess the expertise of mathematical programming, which most database developers do not have. In contrast, in DGQL there is no conceptual separation between data manipulation and optimization. As explained in detail in the fourth and fifth sections, the user needs to only understand the computational process of SQL, plus the concept of "inverse" computation, which is the instantiation of table columns in such a way that all of the database integrity constraints are satisfied, and a particular value computed by SQL is minimized or maximized.

The languages most closely related to DGQL are those that translate procedural algorithms into declarative constraints. These languages unify procedural and constraint semantics, such that the same program statements determine both interpretations. The language Modelica [14] supports unified models that can define both simulation and optimization problems. Modelica models are translated into equations, which may in turn be solved by an optimizer or sorted and compiled into an efficient sequential procedure. Within pure functions (functions without side effects), Modelica can also translate procedural algorithms into constraint equations. Within these functions, Modelica gains the advantage of specifying constraints using familiar procedural operations and flow of control. However, Modelica is fundamentally an equational language, and it supports procedural algorithms only in this limited context.

The language CoJava [6] has a thoroughly procedural object-oriented syntax and semantics of Java, and adds the optimization semantics of an optimal nondeterministic execution path. Thus, CoJava presents the developer with no visible boundary between procedures and constraints. Familiar procedural operations and flow of control can be used uniformly throughout an entire model, or even throughout an entire software system.

DGQL is in the spirit of Modelica and especially CoJava to minimize the learning curve for database application developers and to take advantage of existing database queries by conforming to a well-understood syntax and semantics. While we chose SQL as the base language to be annotated, the same idea of annotating a database

program and translating it to a formal optimization problem can be applied to other database languages, including Query-by-Example (QBE) and object relational query languages such as Hibernate Query Language (HQL). Using this approach, DGQL gives database application developers the flexibility to move model components freely back and forth between query evaluation and declarative optimization models.

Despite the challenges mentioned earlier, DGSS are being built and deployed today. However, their development to date amounts to a large modeling and software development effort. Furthermore, as business requirements evolve, additional development will periodically be needed to incorporate these requirements into the optimization model. DGQL, however, continues the tested database tradition in building a level of abstraction that is closer to the user intuition. By doing so, many applications can take advantage of decision optimization without large up-front investment and heavy long-term maintenance expenses.

DGQL by Example

IN THIS SECTION, WE USE THE EXISTING QUERIES in our logistics example shown in the second section to illustrate the DGQL framework. We discuss the DGQL formal syntax and semantics in the next section. Note that in this paper all database examples are purely SQL with the exception of DGQL annotation that we indicate by **BOLDFACE ITALICS** in the code listings.

Continuing with the example in the second section, we assume that the `orders` table is the one we need to fill, with the objective of minimizing the total cost as described by the `total_cost` query (view). Instead of the given database `orders` table, we replace it with a view created by the following SQL statement plus an `augment` annotation:

```
CREATE VIEW orders AS AUGMENT
    (SELECT d.loc, s.vendor, s.item
FROM demand d
LEFT OUTER JOIN supplier s
ON d.item = s.item)
WITH quantity INTEGER >=0;
```

Note here that the nested query is a standard query, giving all possible pairings of vendor with item for each location, while the view creation here is a DGQL annotation that indicates the resulting table of the nested query to be *augmented* with an extra column called `quantity` with the condition that the quantity has to be a nonnegative integer. The augmented column is the one to be filled by DGQL execution automatically with optimal objective (to be annotated below).

Intuitively, with any assignment of nonnegative values to the augmented `quantity` column to the generated `orders` table, there is a total cost value given by the `total_cost` view. The task now is to automatically generate an assignment that gives the minimum total cost for our logistics example. However, if we examine the `total_cost` view, we note that table `demand` is not used. When the reporting query is written, we have assumed that the `orders` table is given externally and that all the

demands are satisfied by the orders. This latter assumption can be checked by the database with the following constraint:

```
CREATE VIEW requested_vs_delivered AS
  SELECT o.loc, o.item, SUM(o.quantity) AS delivered,
    d.requested
FROM orders o
INNER JOIN demand d
ON o.loc = d.loc AND o.item = d.item
GROUP BY o.loc, o.item, d.requested
CHECK delivered >= requested;
```

This constraint checks if an item request is satisfied at a location with enough orders by comparing the total quantities of an item ordered for the location with the demand quantity at that location. This constraint needs to be explicitly given in order to generate the `orders` table correctly, and DGQL supports the above constraint specification (while implementation of such a constraint in standard DBMS is not uniformly present).

Finally, we need to specify the objective explicitly. In DGQL, we use the following statement:

```
MINIMIZE total_cost;
```

Note that `total_cost` is the view defined in the second section. The semantics of the `MINIMIZE` is to create a table for `orders` (in general, all augmented tables are instantiated) in which the last column is filled with integer values, in the way such that the total cost is minimum among all possible orders to satisfy all the item demands.

The above optimization problem of minimizing the cost can be solved rather easily since the optimal strategy is always to use the least costly supplier for each item. This strategy can be coded directly in SQL and provide optimal solution without using DGQL or MP (although the code will look different from the reporting query, increasing the management complexity). However, in general, the capability of SQL in providing decision optimization is limited. For example, consider the situation that (1) each vendor has a limited supply of each item, and (2) in practice, we want to limit the number of (say up to three) vendors to supply all the items for a particular location in order to facilitate easier tracking and management. In this case, it is not clear how to use SQL to directly code an optimal solution. In DGQL framework, this can be done with the following two additional constraints:

```
CREATE VIEW purchased_vs_available AS
  SELECT o.vendor, o.item, SUM(o.quantity) AS purchased,
    s.available
FROM orders o
INNER JOIN supplier s
ON o.vendor=s.vendor AND o.item=s.item
GROUP BY o.vendor, o.item, s.available
CHECK purchased <= available;
```

```

CREATE VIEW vendor_restriction AS
  SELECT loc, COUNT(*) AS vendors
  FROM (SELECT DISTINCT loc, vendor
        FROM orders o
        WHERE quantity>0)
  GROUP BY loc
  CHECK vendors<=3;

```

The first constraint above states that the ordered item quantity from a vendor does not exceed its available quantity, and the second enforces that the total number of distinct vendors for each location does not exceed three. With the above `CHECK` statements, DGQL will find the optimal solution satisfying these constraints.

Note that a constraint is quite different from a selection (`WHERE`) condition. Indeed, in our example, constraints will limit the possible `orders` table instantiations to be considered in obtaining the optimal solution by rejecting those that do not satisfy the constraints.

In addition, DGQL allows modular development. If shipping and other options are to be considered, we only need to annotate the relevant queries and add appropriate constraints in order to extend the decision optimization problem. DGQL also takes the relevant existing SQL queries without any change in participating in decision optimization, reusing existing investment.

Formal Syntax and Semantics

WE NOW GIVE A PRECISE EXPOSITION OF THE DGQL SYNTAX AND SEMANTICS. The philosophy of DGQL is to add annotation, in the form of operators, to the standard SQL in order to express decision optimization problems. Like the standard SQL, the precise syntax and semantics is best explained in the relational algebra. Hence, in this section we use the approach of adding a number of operators to the standard relational algebra. The informal, SQL-like expressions directly correspond to these operators.

Like the standard relational algebra, we assume a given set of base relations in a database, each having a unique name and schema. We also assume there is a supply of globally unique relation names to be used for assignments, each with a schema. We assume the conventional notions and notation (e.g., [16]), including those of schemas, domains, tuples, relations, databases, and all the traditional algebraic operations, as well as the assignment statement (or view definition). We also use a simple form of aggregation (as the “aggregation formulation” in [25]).

The annotation of DGQL consists of three operations: augmentation, constraint, and optimization. All three operators are used in the example in the previous section. Honoring the tradition, except for the optimization operator, which is the last statement in any DGQL program, we define augmentation and constraint as relational operators that can appear at any step of a relational expression.

For the sake of completeness, we list all the operators that can be used in DGQL. More precisely, we define DGQL query expressions recursively as follows:

1. (Base Case) Each relation name (base relation or not) is a DGQL query expression. The schema of the expression is exactly the relation schema of the base relation.
2. (Traditional Operators) Given DGQL query expressions e_1 and e_2 , then $\pi_L(e_1)$, $\sigma_C(e_1)$, $e_1 \times e_2$, $e_1 \cup e_2$, $e_1 \cap e_2$, and $e_1 - e_2$ are all DGQL query expressions. The operations are all the traditional relational operators, and we assume the schema requirement of the operators are satisfied by e_1 and e_2 . The schemas of the expressions are as defined traditionally.
3. (Arithmetic) Given DGQL query expression e , then $\gamma[B=arith(L)](e)$, where L is a subset of numeric attributes in e , B is a new attribute, and $arith(L)$ is an arithmetic expression using attributes in L .
4. (Aggregation) Given DGQL query expression e and assuming $A_1, \dots, A_n$ and B are attributes of the schema of e , then $\delta[A_1, \dots, A_n, f(B)](e)$ is a DGQL query expression, where f is an aggregation function (mapping bags to values). The schema of the expression is $A_1, \dots, A_n, f(B)$. A special case is when $n = 0$, in which only $f(B)$ attribute exists for the schema of the expression.
5. (Augmentation from Relation) Given DGQL query expressions e_1 and e_2 and assuming $A_1, \dots, A_n$ are attributes in e_1 and not attributes in e_2 , $\alpha[A_1, \dots, A_n \in e_1](e_2)$ is also a DGQL query expression. The schema of the expression is the schema of e_2 extended with $A_1, \dots, A_n$.
6. (Augmentation from Domain) Given DGQL query expression e , $\alpha[A \in domain](e)$ is also a DGQL expression, where $domain$ is a set of values of a specific type, such as $real[0, \infty)$ for nonnegative reals. This form of augmentation is equivalent to the above one if $domain$ is taken as a (possibly infinite) relation with one attribute A and each value is taken as a tuple.
7. (Constraint) Given DGQL query expression e , then $\xi[C](e)$ is a DGQL query expression, where C a condition as found in selection. The schema of the expression is that of e .
8. (Optimization) Given a DGQL query expression e that computes a 1-tuple relation over a schema with a single attribute, $\omega[\min/\max](e)$ is also a DGQL query expression. The schema of the expression is that of e .

Definition 1: A DGQL query is an assignment sequence of the form

$$R_1 = e_1; \dots; R_n = e_n; \omega,$$

where each e_i is a DGQL expression and ω is either $\min(R_k)$ or $\max(R_k)$, where $1 \leq k \leq n$ and e_k must compute a single numeric value, or sat . That is, a DGQL query is a sequence of assignments followed by an optimization operator or a satisfiability operator.

A *valid* DGQL query is one that always uses a relation name that is either a base relation name or one that has been defined (i.e., on the left-hand side of an assignment) earlier in the sequence. We henceforth consider only valid DGQL expressions.

The example in the fourth section can be easily rewritten in the DGQL algebraic query form, with each view being an assignment followed by $\omega = \min(\text{total_cost})$.

We now define the semantics of DGQL queries. For simplicity, we can assume that in a DGQL query, each e_i is either a single relation name (base relation name or one of $R_1, \dots, R_{i-1}$) or a single operator applied to operands in the form of single relation names. In other words, each assignment is a “simple step” of copying a relation or applying one operation over the previously defined relations. We also assume the optimization operator only applies to single relation name, such as $\omega(R_k)$ where $1 \leq k \leq n$.

Definition 2: A DGQL query is a *basic assignment sequence* if each assignment is a simple step as described above.

Since a DGQL query can be rewritten into a basic assignment sequence by introducing more assignments, without loss of generality, we henceforth assume all DGQL queries are basic assignment sequences if not mentioned otherwise.

Intuitively, (1) the assignment of a DGQL query is to give all “feasible” instantiations of all the relations for the left-hand side of each assignment such that the “input” and “output” of each assignment is “correct,” that is, given the input relations, the left-hand side relation is actually the result of the operator applied to the given operands; (2) the operator ω as the last step of a DGQL query is to find one such feasible instantiation, when ω is *sat*, with the additional restriction that the e_k value is the minimum or maximum among all the feasible instantiations, when ω is either $\min(R_k)$ or $\max(R_k)$, respectively. For example, the DGQL query in the fourth section obtains an assignment to all the views, including the view *orders* such that the total cost is minimized. In this way, the optimization done by DGQL automatically fills the *orders* table for the logistics optimization application.

Different from the standard SQL, where the operands and the operator determine the output relation, the DGQL operator augmentation introduces nondeterminism; that is, for the augmentation operator, infinitely many “output” relations may correspond to an “input” relation. Therefore, a DGQL query can have many feasible instantiations for the assignment sequence. And the optimization operator in the end is to choose one feasible instantiation.

Formally, consider a DGQL query $R_1 = e_1; \dots; R_n = e_n; \omega$. A (nondeterministic) *execution path* is a sequence of relations $(r_1, \dots, r_n)$ over the schemas of $e_1, \dots, e_n$, respectively. A *partial execution path* is a prefix of an execution path, that is, a sequence of relations $(r_1, \dots, r_i)$, $1 \leq i \leq n$, over the schemas of $e_1, \dots, e_i$, respectively, or the empty prefix ϵ .

Definition 3: The *feasibility* of a partial execution path is defined recursively as follows. The empty execution path ϵ is feasible. Recursively, $(r_1, \dots, r_i)$, $1 \leq i \leq n$, is feasible if (1) $(r_1, \dots, r_{i-1})$ is feasible and (2) the following are satisfied:

- if e_i is a *traditional* or *aggregation* operation, then r_i is the result of e_i computation when operands from $R_1, \dots, R_{i-1}$ are replaced with relations $r_1, \dots, r_{i-1}$, respectively;
- if e_i is an augmentation operation $\alpha[A_1, \dots, A_m \in R_j](R_k)$, where $1 \leq k, j < i$, r_i has the schema of R_k extended with $A_1, \dots, A_m$, and is constructed by joining a tuple from r_k with a single tuple (nondeterministically chosen) from $\pi_{A_1, \dots, A_m}(r_j)$. Similarly, if e_i is $\alpha[A \in \text{domain}](R_k)$, where $1 \leq j < i$, r_i has the

- schemas of R_k extended with A , and is constructed by joining a tuple from r_k with a single value (nondeterministically chosen) from the *domain* of A ;
- if e_i is a constraint operation $\xi[C](R_j)$, $1 \leq j < i$, the constraint C is satisfied by r_j and $r_i = r_j$.

We say $(r_1, \dots, r_n)$ is a *feasible execution path* if it is a *feasible* partial execution path and denote by $F(d)$ the set of all feasible execution paths for $R_1 = e_1; \dots; R_n = e_n$ and input database d .

Definition 4: The *semantics* of a DGQL query

$$R_1 = e_1; \dots; R_n = e_n; \omega$$

is defined as the function

$$\Phi : D_1 \rightarrow 2^{D_2},$$

where D_1 is the set of all relational databases over the base database schemas, and D_2 is the set of all databases over the schema $S_1, \dots, S_n$ of the expressions $e_1, \dots, e_n$, respectively, as follows. For a given database d , $\Phi(d)$ is the set of all databases $d' = (r_1, \dots, r_n)$ that satisfy the ω operator, that is,

Case 1: When $\omega = \text{sat}$, $\Phi(d)$ is the set of all feasible execution paths, i.e., $\Phi(d) \stackrel{\text{def}}{=} F(d)$.

Case 2: When $\omega = \min(R_k)$, $\Phi(d)$ is the set of all feasible execution paths that give the minimum value computed by r_k among all feasible execution paths, i.e., $\Phi(d) \stackrel{\text{def}}{=} \{(r_1, \dots, r_n) \in F(d) \mid \forall (r'_1, \dots, r'_n) \in F(d), r_k \leq r'_k\}$.

Case 3: When $\omega = \max(R_k)$, $\Phi(d)$ is the set of all feasible execution paths that give the maximum value computed by r_k among all feasible execution paths, i.e., $\Phi(d) \stackrel{\text{def}}{=} \{(r_1, \dots, r_n) \in F(d) \mid \forall (r'_1, \dots, r'_n) \in F(d), r_k \geq r'_k\}$.

Note that the optimal value may not lead to a unique feasible execution path, that is, more than one assignment to relations $r_1, \dots, r_n$ may lead to the same optimal value for r_k . We say that a database $d' = (r_1, \dots, r_n) \in \Phi(d)$ is a nondeterministic answer to the DGQL query. If $\Phi(d) = \emptyset$, we say that $\perp$ (to denote infeasibility) is returned and assigned to $R_1, \dots, R_n$.

DGQL Reduction to MP

IN THIS SECTION, WE PRESENT A REDUCTION PROCEDURE to translate DGQL queries to MP formulations. We first introduce MP with some discussion, and then give a *sound* and *complete* reduction procedure.

MP and Its Computational Complexity

An MP problem has the form $\min f(\hat{x})$ subject to $C(\hat{x})$, where $\hat{x}$ is a vector of variables that range over domain $\hat{D}$, f is an *objective* function from $\hat{D}$ to the set of reals

$\mathfrak{R}$, and $C(\hat{x})$ is a constraint. The constraint $C(\hat{x})$ is associated with a Boolean function that, given an instantiation of vector $\hat{a} \in \hat{D}$ to $\hat{x}$, evaluates to true or false. A solution to the optimization problem is a vector $\hat{a} \in \hat{D}$ that satisfies the constraint C so that $f(\hat{a}) \leq f(\hat{b})$ for all vectors $\hat{b}$ that satisfy C . Note that $\min$ can be replaced with $\max$ in this formulation without difficulties.

Formal optimization models are solved using MP algorithms. MP algorithms heavily depend on the type of the problem, determined by the domains of the variables (e.g., reals, integers, binary, finite domain), the structure of constraints C (e.g., linear, quadratic, polynomial), and the form of the objective function f . As an example, linear programming (LP) is a class of MP where variables range over reals, constraints are inequalities between linear arithmetic expressions, and the objective function is linear.

The LP problem was proven to be in P in Kachiyan [21]. However, the class of simplex algorithms [10] is still the most widely used in practice, although its worst-case complexity is exponential in the dimension (i.e., the number of variables). But even the $\{0, 1\}$ integer linear programming (ILP) is known to be NP-complete [27]. The Branch and Bound algorithm is a popular algorithm for ILP and MILP (mixed-integer LP). While exponential in the worst case, it has the nice property of reaching intermediate (not necessarily optimal) solutions. In this paper, we use MILP to encode DGQL queries and derive solutions.

Reduction Procedure

The key idea of the reduction procedure is to use a symbolic table formulation to represent each step of the DGQL query. Without loss of generality, we assume that every tuple in a relation has a globally unique tuple identifier *tid* and that when new tuples are introduced (e.g., by materializing a view definition), new *tids* will be generated accordingly.

Definition 5: A *symbolic table* is a standard relational table with the following additions: (1) an attribute can be of a constraint variable name type (or CVN type), and the domain of a CVN attribute is variable names, and (2) there must be a special attribute of CVN type named TAF, for tuple-active-flag, with domain of binary variable names.

As an example, below is a symbolic orders table:

TAF	sup	item	loc	qty
taf[1]	S1	I1	L1	qty[1]
taf[2]	S2	I2	L2	qty[2]

where TAF and qty are CVN-type attributes. The purpose of the TAF attribute is to indicate that a particular tuple exists in the table in a feasible execution path, and the CVN attributes allow possible instantiations.

Symbolic tables can be instantiated as regular relational tables by instantiating all the variables with CVN attributes. More specifically, given an instantiation of

variables appearing in a symbolic table, the symbolic table becomes a regular table, minus the `TAF` column, as follows: For each tuple t in the symbolic table, if $\text{taf}[id]$ is instantiated to `True`, then the resulting regular table contains the tuple t with other CVN attributes instantiated and the `TAF` attribute dropped. For example, in the above table, if $\text{taf}[1] = \text{True}$, $\text{taf}[2] = \text{False}$, $\text{qty}[1] = 20$, and $\text{qty}[2] = 4$, then the resulting table is

sup	item	loc	qty
S1	I1	L1	20

Note that the `TAF` attribute is dropped.

The reduction to MILP is as follows. Given a DGQL query $R_1 = e_1; \dots; R_n = e_n; \omega$, we generate a symbolic table for each R_i . In order to represent all the feasible execution paths, constraints are to be added so all and only feasible execution paths are represented. More specifically, the *reduction result* will be a triple:

$$(\{s_1, \dots, s_n\}, V, C),$$

where $\{s_1, \dots, s_n\}$ are symbolic relational tables, V is the set of all constraint variables, including those that appear in $\{s_1, \dots, s_n\}$, and C is a set of constraints over V . The schema of each s_i , $1 \leq i \leq n$, will be the schema of e_i extended with the `TAF` attribute.

Definition 6: Given the reduction result $(\{s_1, \dots, s_n\}, V, C)$ and an instantiation of variables in V to values in their corresponding domains, we say that $(r_1, \dots, r_n)$ is a *reduction instantiation* of $(\{s_1, \dots, s_n\}, V, C)$ if $\forall i$ such that $1 \leq i \leq n$, r_i is constructed by (1) replacing variables in s_i with their corresponding instantiated values, (2) eliminating all tuples in s_i where the `TAF` attribute is `False`, and (3) removing the `TAF` attribute.

Before we discuss the procedure to obtain a reduction from a DGQL query, we formalize the correctness of the reduction result. As discussed above, the symbolic tables together with the constraints should represent all and only feasible execution paths. To formalize this idea, we have:

Definition 7: The reduction result $(\{s_1, \dots, s_n\}, V, C)$ of a DGQL query $R_1 = e_1; \dots; R_n = e_n; \omega$ is said to be *correct* if:

- *Soundness:* Every reduction instantiation $(r_1, \dots, r_n)$ of $(s_1, \dots, s_n)$ that satisfies constraint C is indeed a feasible execution path of the DGQL query; and
- *Completeness:* Every feasible execution path is a reduction instantiation of $(s_1, \dots, s_n)$ that satisfies C .

The reduction steps follow the traditional relational algebra approach in the sense that at each step we simply generate a new symbolic table and add appropriate constraints. These steps are summarized in Algorithm 1.

Algorithm 1 produces symbolic tables s_i recursively using the symbolic tables generated earlier. The base case (steps 2–5) is to make the base database tables symbolic by adding a `TAF` attribute and the corresponding variables and constraints, indicating that

Algorithm 1. Constraint Reduction

Input: A DGQL query $R_1 = e_1; \dots; R_n = e_n; \omega$
Output: A correct reduction $(\{s_1, \dots, s_n\}, V, C)$
Method:
1: Let $V = \emptyset$ and $C = \text{True}$
2: **for** each base table B **do**
3: generate a symbolic table by adding a TAF attribute
4: for each tuple in B with tuple id tid , use $CVN_{\text{taf}[tid]}$ as the value under TAF, and let $C = C \wedge \text{taf}[tid]$. Add $\text{taf}[tid]$ to V .
5: **end for**
6: **for** $i = 1$ to $n - 1$ **do**
7: Generate symbolic table s_i from the operator of e_i and the input symbolic tables used in e_i (these input symbolic tables must be from $s_1, \dots, s_{i-1}$ and those corresponding to the base tables).
8: Generate the corresponding C_i
9: Add the new variables used in s_i and C_i to V
10: Let $C = C \wedge C_i$
11: **end for**
12: **return** $(\{s_1, \dots, s_n\}, V, C)$

all the tuples are “active” in the base tables. At each step of the reduction (steps 7–10), a new symbolic table for e_i is generated and constraints and variables added to C and V , respectively. We now describe the key idea behind these steps. In Appendix A, we give a complete description of these steps for each operator.

Step 7 of Algorithm 1 is to generate a symbolic table for s_i given one or two symbolic tables as the input. The schema of the output table is easily found by using the formal definition of the DGQL and traditional relational algebra operators. Setting up all the tuples in s_i is easy for most operators, but a bit challenging for others. Intuitively, we want to set up all the possible output tuples in s_i and use constraints involving $\text{taf}[tid]$ to indicate if the tuple should actually exist in a feasible execution path. For example, if the operator is the selection operation, then the output table can be simply a copy of the input table, but in the constraint we code that an output tuple exists only if the selection condition is true. The reason we need to use constraint instead of simply dropping the tuple is that the selection condition may be applied to a CVN entry whose value is unknown at the reduction time.

The number of all possible tuples in s_i can be exponential in some cases. For example, when “group by” is done on a CVN attribute, we need to set up s_i to prepare for all possible groupings since we do not yet know the values of these CVN attributes. In practice, we may want to restrict “group by” only on non-CVN attributes so that the grouping will be known at the reduction time and each group will correspond to one possible tuple in s_i (although the actual values in the tuple may remain CVN and unknown at the reduction time). Details can be found in Appendix A.

Algorithm 2. DGQL Computation

Input: A DGQL query $R_1 = e_1; \dots; R_n = e_n; \omega$ and a database d

Output: Query result

Method:

1. Perform Constraint Reduction (Algorithm 1) to produce $(\{s_1, \dots, s_n\}, V, C)$
 2. **if** constraint C is infeasible **then**
 3. **return** $\perp$
 4. **else**
 5. **if** $\omega = \text{sat}$ **then**
 6. Solve the CSP for C and V
 7. **else**
 8. Solve the MP problem $\min/\max V_{R_k}$ such that C is satisfied
 9. **end if**
 10. Given the variable instantiation produced in Step 6 or Step 8, return the Reduction Instantiation $(r_1, \dots, r_n)$ as described in Definition 6.
 11. **end if**
-

Once the symbolic output table is set up, the remaining task is to generate the constraint (Step 8). This step is basically to encode the correspondence between the input tuples and the possible output tuples. Since we know which symbolic tuple in the output is generated by which input tuples, this task is relatively easy as this is in reality a tuple-level task. Again, we refer the reader to Appendix A for further details.

We are now ready to address the correctness of the reduction.

Theorem 1: The reduction produced by Algorithm 1 is correct, that is, both sound and complete.

Using the correct reduction, the result of a DGQL query can be derived as indicated in Algorithm 2. Basically, we solve the corresponding MP problem given by the correct reduction result (Step 1). If the constraint is infeasible, then we return $\perp$ (Steps 2–3). Otherwise, we use the instantiation of the variables found in the constraint satisfaction problem (CSP) solution (Step 6) or MP solution (Step 8) to instantiate all the symbolic tables (Step 10).

Theorem 2: Algorithm 2 correctly, by Definition 4, computes the DGQL query result.

Initial Experimental Evaluation

THE RUNNING TIMES OF MATHEMATICAL PROGRAMS IS OFTEN SENSITIVE to the way the problem is represented. For many classes of problems, a poor choice of mathematical representation will cause the solver to take significantly longer to find an optimal solution. One concern with DGQL (and the reduction process as described in the previous section)

is that the extra views and tables used in modeling add substantial overhead by introducing redundant or unnecessary variables and constraints. It is possible a simpler or more concise modeling may lead to a significantly faster solution.

Our hypothesis is that problems described using DGQL are solved as efficiently as when described using algebraic modeling languages such as AMPL or GAMS. As an initial step to test this hypothesis, we took the logistics optimization problem from the second section and manually created a concise AMPL formulation of the same problem. We then generated several database instances varying problem size and compared the running time when solved using this AMPL model and through DGQL reduction. We now present the complete AMPL implementation of our running example:

```

set Vendors;
set Items;
set Locations;
param PricePerUnit{Vendors,Items}>=0;
param Available{Vendors,Items}>=0;
param Requested{Locations,Items}>=0;
var Orders{Vendors,Items,Locations}>=0;
var OrderPlaced{Vendors,Items,Locations} binary;
var ItemsPurchased{Vendors,Locations}>=0;
var VendorShipped{Vendors,Locations} binary;

minimize total_cost:
    sum{v in Vendors} sum{i in Items} sum{l in Locations}
        PricePerUnit[v,i] * Orders[v,i,l];

subject to
    available_vs_purchased {v in Vendors, i in Items}:
        Available[v,i] >= sum{l in Locations} Orders[v,i,l];
    requested_vs_delivered {l in Locations, i in Items}:
        Requested[l,i] <= sum{v in Vendors} Orders[v,i,l];
    order_placed {v in Vendors, i in Items, l in Locations}:
        OrderPlaced[v,i,l] = 0 ==>
            Orders[v,i,l] = 0 else Orders[v,i,l] >= 1;
    items_purchased {v in Vendors, l in Locations}:
        ItemsPurchased[v,l] = sum{i in Items} OrderPlaced[v,i,l];
    vendor_shipped {v in Vendors, l in Locations}:
        VendorShipped[v,l] = 0 ==>
            ItemsPurchased[v,l] = 0 else ItemsPurchased[v,l] >= 1;
    vendor_restriction {l in Locations}:
        sum{v in Vendors} VendorShipped[v,l] <= 3;

```

The DGQL query and the AMPL program were run using ILOG CPLEX 12.1 as the solver. The test was on Windows XP running on a 2.0GHz Core 2 Duo with 4GB of RAM. In both cases, the tuning parameters passed to ILOG CPLEX were the same and were typical for solving MILP problems.

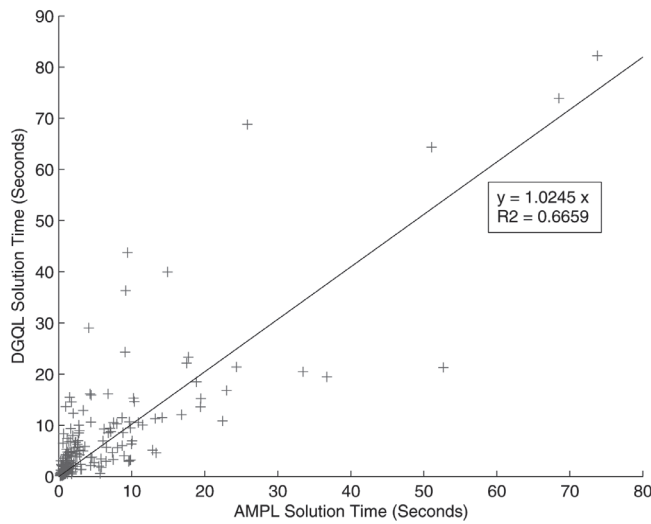


Figure 1. Scatter Plot of Solution Time (DGQL Versus AMPL)

The size of the logistics optimization problem here is parameterized by the number of vendors, items, and locations. Given these parameters, a problem is generated by randomly instantiating the database tables `supply` and `demand`. Note that problem size corresponds to the number of decision variables in `orders` as well as the indicator variables (tuple-active-flags) implicit in `vendor_restrictions`. Thus, we compute this directly from the problem parameters, that is, *decision variables* = *vendors* * *items* * *locs* and *indicator variables* = *vendors* * *items* * *locs* + *vendors* * *locs*.

Each random problem generated was solved using both the DGQL reduction and the AMPL program, and the solution time was measured. This is the CPU time spent searching for an optimal solution and does not include the time to load the program off disk, or in the case of DGQL from the database. Only random databases that contained a feasible solution were included in the results. The solution time for both the DGQL and AMPL implementations was less than 0.01 seconds for every randomly generated infeasible problem. The results of this experiment can be seen in Figure 1.

Note in the graph, a linear regression through the origin has slope 1.0245. This implies the AMPL model is on average ≈ 2.45 percent faster than DGQL. This difference is small and not significant when considering the large variation in solution time between the two systems for the same problem.

On closer inspection, it appears the advantage AMPL has is limited only to those problem instances where a tight solution is found by CPLEX quickly. Our DGQL implementation uses Java and Java Native Interface to communicate with CPLEX, and this may add overhead to small problems. Moreover, AMPL performs limited presolving when converting problems into a special format that CPLEX understands, which may reduce CPLEX solution time.

In addition, DGQL appears to do better on problem instances where a larger parameter space must be searched. This is due in part to the fact that the AMPL formulation

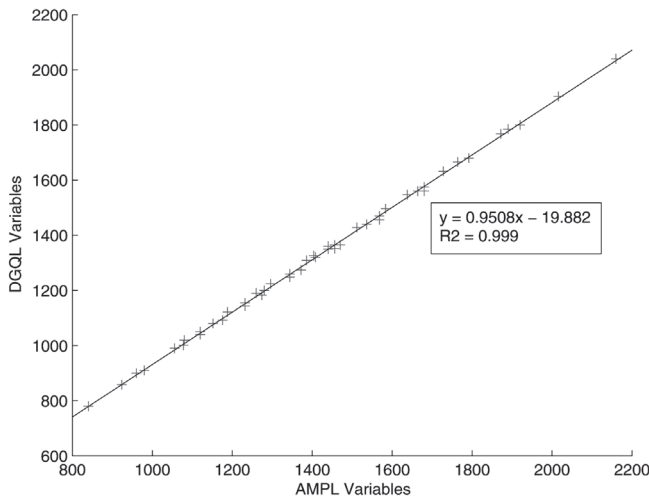


Figure 2. Scatter Plot of Total Number of Variables (DGQL Versus AMPL)

has an extra *vendors * locs* number of variables in its solution space (see Figure 2). As the logistics optimization problems become larger, this extra number of variables will have a measurable impact on solution time. It will be interesting to investigate further if these extra variables are a limitation of the AMPL modeling language or just our formulation of this problem.

Conclusion

IN THIS PAPER, WE PROPOSED DGQL, a language designed to (re-)use SQL programs for decision optimization to make DGSS implementation agile and intuitive and leverage existing investment in SQL-implemented MIS. We defined its formal syntax and semantics and provided a canonical implementation of DGQL programs by reduction to mathematical programming. We used examples to show that the overhead of using a high-level language is small when compared with manually crafted MP formulations. With added benefit of agile and intuitive high-level language, we believe that DGQL provides a practical solution for decision optimization in DGSS, especially when an underlying database is involved and database applications already exist.

As a database language, DGQL opens up many research questions. For example, what is the expressiveness of DGQL: how to characterize it, and is there a completeness as in relational algebra. Optimization opportunities are abundant and we only touched on a few during our canonical reduction process. Standard database techniques may help, but more interesting is how to use the high-level nature of the language to help find optimal solutions, as it is well known in the database community that more semantic information usually translates to better query optimization. Additional experimental study will be important both in terms of DGQL efficiency for a wide variety of MP models. Finally, it will be important to conduct usability studies on complete business

cases to measure productivity gained by using DGQL as opposed to algebraic modeling languages integrated with a relational database management system directly.

Acknowledgment: Part of the work by Wang was supported by National Science Foundation grant no. 0716575.

REFERENCES

1. Agerfalk, P.J.; Fitzgerald, B.; and Slaughter, S.A. Introduction to the special issue—Flexible and distributed information systems development: State of the art and research challenges. *Information Systems Research*, 20, 3 (2009), 317–328.
2. Atamtürk, A.; Johnson, E.L.; Linderöth, J.T.; and Savelsbergh, M.W.P. A relational modeling system for linear and integer programming. *Operations Research*, 48, 6 (November–December 2000), 846–857.
3. Beck, K. Embracing change with extreme programming. *Computer*, 32, 10 (1999), 70–77.
4. Bhargava, H.K.; Krishnan, R.; and Mukherjee, S. On the integration of data and mathematical modeling languages. *Annals of Operations Research*, 38, 1 (November–December 1992), 69–95.
5. Boisvert, R.F.; Howe, S.E.; and Kahaner, D.K. GAMS: A framework for the management of scientific software. *ACM Transactions on Mathematical Software*, 11, 4 (1985), 313–355.
6. Brodsky, A., and Nash, H. CoJava: Optimization modeling by nondeterministic simulation. In F. Benhamou (ed.), *Proceedings of Principles and Practice of Constraint Programming (CP 2006)*, vol. 4204. Nantes, France: Springer, 2006, pp. 91–106.
7. Capiluppi, A.; Fernandez-Ramil, J.; Higman, J.; Sharp, H.C.; and Smith, N. An empirical study of the evolution of an agile-developed software system. In *Proceedings of the 29th International Conference on Software Engineering (ICSE '07)*. Los Alamitos, CA: IEEE Computer Society, 2007, pp. 511–518.
8. Choobineh, J. SQLMP: A data sublanguage for representation and formulation of linear mathematical models. *INFORMS Journal on Computing*, 3, 4 (Fall 1991), 358–375.
9. Cohen, D.; Lindvall, M.; and Costa, P. An introduction to agile methods. In M. Zekowitz (ed.), *Advances in Computers*, vol. 62. Oxford: Elsevier, 2004, pp. 1–66.
10. Dantzig, G. *Linear Programming and Extensions*. Princeton: Princeton University Press, 1963.
11. Desouza, K.C. *Agile Information Systems: Conceptualization, Construction, and Management*. Oxford: Elsevier, 2006.
12. Dolk, D.R. An introduction to model integration and integrated modeling environments. *Decision Support Systems*, 10, 3 (1993), 249–254.
13. Fourer, R.; Gay, D.M.; and Kernighan, B.W. A modeling language for mathematical programming. *Management Science*, 36, 5 (1990), 519–554.
14. Fritzson, P., and Engelson, V. Modelica—A unified object-oriented language for system modeling and simulation. In E. Jul (ed.), *Proceedings of the 12th European Conference on Object Oriented Programming (ECOOP 98)*. London: Springer, 1998, pp. 67–90.
15. Fruhling, A., and Vreede, G.-J. de. Field experiences with eXtreme programming: Developing an emergency response system. *Journal of Management Information Systems*, 22, 4 (Spring 2006), 39–68.
16. Garcia-Molina, H.; Ullman, J.D.; and Widom, J. *Database Systems: The Complete Book*, 2d ed. Englewood Cliffs, NJ: Prentice Hall, 2008.
17. Gremillion, L.L., and Pyburn, P.J. Justifying decision support and office automation systems. *Journal of Management Information Systems*, 2, 1 (Summer 1985), 5–17.
18. Harris, M.L.; Collins, R.B.; and Hevner, A.R. Control of flexible software development under uncertainty. *Information Systems Research*, 20, 3 (2009), 400–419.
19. Hong, W.; Thong, J.Y.L.; Chasalow, L.C.; and Dhillon, G. User acceptance of agile information systems: A model and empirical test. *Journal of Management Information Systems*, 28, 1 (Summer 2011), 235–272.

20. Ilyas, I.F.; Beskales, G.; and Soliman, M.A. A survey of top-k query processing techniques in relational database systems. *ACM Computing Surveys*, 40, 4 (2008), 1–58.
21. Kachiyani, L.G. A polynomial algorithm in linear programming. *Soviet Mathematics Doklady*, 20, 1 (1979), 191–194.
22. Lindstrom, L., and Jeffries, R. Extreme programming and agile software development methodologies. *Information Systems Management*, 21, 3 (2004), 41–52.
23. Maruping, L.M.; Venkatesh, V.; and Agarwal, R. A control theory perspective on agile methodology use and changing user requirements. *Information Systems Research*, 20, 3 (2009), 377–399.
24. Mitra, G.; Lucas, C.; Moody, S.; and Kristjansson, B. Sets and indices in linear programming modelling and their integration with relational data models. *Computational Optimization and Applications*, 3, 4 (1995), 263–283.
25. Özsoyoğlu, G.; Özsoyoğlu, Z.M.; and Matos, V. Extending relational algebra and relational calculus with set-valued attributes and aggregate functions. *ACM Transactions on Database Systems*, 12, 4 (1987), 566–592.
26. Rumpe, B., and Schröder, A. Quantitative survey on extreme programming projects. In M. Marchesi and G. Succi (eds.), *Proceedings of the 3rd International Conference on Extreme Programming and Agile Processes in Software Engineering (XP 2002)*. Alghero, Italy: Springer, 2002, pp. 95–100.
27. Schrijver, A. *Theory of Linear and Integer Programming*. New York: John Wiley & Sons, 1998.
28. Thomas, D. Agile programming: Design to accommodate change. *IEEE Software*, 22, 3 (May–June 2005), 14–16.
29. Vidgen, R., and Wang, X. Coevolving systems and the organization of agile software development. *Information Systems Research*, 20, 3 (2009), 355–376.

Appendix A: Reduction Procedure

THE KEY IDEA BEHIND CONSTRAINT ENCODING of the step $R_i = e_i$ of a DGQL sequence $R_1 = e_1; \dots; R_n = e_n$; ω depends on the algebraic operator in e_i . For an *augment* operator, the idea is to replace an augmented attribute in every tuple with a CVN attribute that will hold a constraint variable name (unique per tuple id). For the *constraint* operator, the idea is to extend the constraints C being built in the reduction with the corresponding constraints over variables or constants in every tuple. For all the *deterministic* operators, the idea is to build a symbolic representation of the result and add the constraint that connects the variables in the output to the variables in the input. For the *optimization* operator, the idea is to represent an optimization problem that minimizes or maximizes the variable value computed by the operand of the optimization operator.

We now present our reduction formally. We will build the reduction iteratively, by constructing a triple $Q_i = (S_i = \{s_1, \dots, s_i\}, V_i, C_i)$, for every i , $1 \leq i < n$, which will be a reduction from the partial assignment sequence expression $(R_1 = e_1; \dots; R_i = e_i)$, as follows. Initially, $S_0 = \emptyset$, $V_0 = \emptyset$, and $C_0 = \text{True}$. Iteratively, given Q_{i-1} , $1 \leq i < n - 1$, Q_i is constructed based on the query expression e_i as follows:

- If e_i is a base relational name R , and r is the corresponding base relation, s_i will be constructed from r by adding the attribute TAF (whose value for a tuple will be the variable $\text{TAF}[\text{tid}]$, where tid is that tuple's id). V_i will be the union of V_{i-1} and the set of new $\text{TAF}[\text{tid}]$ variables. C_i will be the conjunction of C_{i-1} and the equality constraint $\text{TAF}[\text{tid}] = \text{True}$ for every tuple in r .

- If e_i is a previously assigned relational name R_j ($1 \leq j < i$), then Q_i is constructed by taking $V_i = V_{i-1}$, $C_i = C_{i-1}$, and $s_i = s_j$.
- If e_i is an augmentation-from-domain $\alpha[A \in D](R_j)$, where $1 \leq j < i$, then Q_i is constructed as follows. The relation s_i is constructed from s_j by adding a CVN(D) attribute, and using as its value the variable name $A[tid]$ for every tuple in s_j . V_i is the union of V_{i-1} with the set of all new variables in the CVN(D) attribute. Finally, $C_i = C_{i-1}$.
- If e_i is an augmentation-from-relation $\alpha[A_1, \dots, A_m \in R_j](R_k)$, where $1 \leq j, k \leq n$, then Q_i is constructed as follows. The relation s_i is constructed from s_k , by adding attributes $A_1, \dots, A_m$ of type CVN (over the corresponding domains in s_j). V_i is the union of V_{i-1} and the set of all new variables. Finally, C_i is the conjunction of C_{i-1} and the constraint

$$\bigwedge_{tid \in Tid} (TAF[tid] = True \rightarrow (A_1[tid], \dots, A_m[tid]) \in s_j),$$

where Tid is the set of all tuple ids in s_k .

- If e_i is a projection operation $\pi_L(R_j)$, where $1 \leq j < i$, then Q_i is constructed as follows. $V_i = V_{i-1}$, $C_i = C_{i-1}$, and $s_i = \pi_{L'}(s_j)$, where $L' = L \cup \{TAF\}$.
- If e_i is a (bag) union operation $R_j \cup R_k$, where $1 \leq k, j < i$, then Q_i is constructed by taking $s_i = s_j \cup s_k$, $V_i = V_{i-1}$, and $C_i = C_{i-1}$.
- If e_i is an intersection operation $R_j \cap R_k$, where $1 \leq j, k < i$, then Q_i is constructed as follows. Assume $A_1, \dots, A_l, B_1, \dots, B_m$ is the schema of both R_j and R_k , where $A_1, \dots, A_l$ are all attributes that are non-CVN in both s_j and s_k . For every pair of tuples $tid_1 \in s_j$ and $tid_2 \in s_k$ that agree on $A_1, \dots, A_l$, construct a tuple tid (new id), with values for $A_1, \dots, A_l$ taken from tid_1 (which are the same as in tid_2), and the values for CVN attributes $B_1, \dots, B_m$ being variable names $B_1[tid], \dots, B_m[tid]$, as well as CVN($\{0, 1\}$) attribute TAF . s_i is constructed as the bag of all such tuples. V_i will be the union of V_{i-1} , the set of all new (globally unique) variables above, and the set of an auxiliary CVN($\{0, 1\}$) variables $E[tid]$ for every tid in s_i . To construct C_i , create the following new constraint for every tuple tid in s_i :

$$E[tid] = True \rightarrow (tid_1 \cdot B_1 = tid_2 \cdot B_1 \wedge \dots \wedge tid_1 \cdot B_m = tid_2 \cdot B_m) \\ \wedge TAF[tid] = TAF[tid_1] \wedge TAF[tid_2] \wedge E[tid],$$

where tid_1, tid_2 are two tuples that gave rise to tuple tid in s_i . Note that values of the B attributes (e.g., $tid_1 \cdot B_1$) may either be drawn from the original domain of B (e.g., if B_1 is a non-CVN attribute in s_j), or a variable name (e.g., $B_1[tid_1]$, if B_1 is a CVN attribute in s_j). Finally, C_i is constructed as the union of C_{i-1} and the set of all new constraints above.

- If e_i is a set difference operation $R_j - R_k$, where $1 \leq j, k < n$, then Q_i is constructed by taking $s_i = s_j - s_k$, $V_i = V_{i-1}$, and $C_i = C_{i-1}$. Note we assume that the set difference cannot involve augmented attributes.
- If e_i is a Cartesian product $R_j \times R_k$, where $1 \leq j, k < i$, then Q_i is constructed as follows. s_i is constructed by (1) computing $s_j \times s_k$, and (2) replacing two attributes $R_j \cdot TAF$ and $R_k \cdot TAF$ with a single attribute TAF of type CVN($\{0, 1\}$). Recall

that each tuple in the Cartesian product will have a new global tid , and then the value of TAF for that tuple will be the variable $TAF[tid]$. V_i is the union of V_{i-1} and the set of all new TAF variables. C_i is the conjunction of C_{i-1} and the constraints $TAF[tid] = TAF[tid_1] \wedge TAF[tid_2]$, for all tid in the Cartesian product, where tid_1 and tid_2 are the ids of tuples from s_j and s_k , respectively, that formed the tuple tid .

- If e_i is selection operation $\sigma_P(R_j)$, where $1 \leq j < i$ and P is the selection predicate, then Q_i is constructed as follows. Let $P = P_{reg} \wedge P_{cvn}$, where P_{reg} is the selection predicate that does not involve CVN attributes, and P_{cvn} is the selection predicate that may involve CVN attributes. s_i is constructed as $\sigma_{P_{reg}}(s_j)$. Note that s_i has globally new tuple ids, and thus the variables $TAF[tid]$ in tuples of s_i are new unique variables. For every new variable $TAF[tid]$, we also create an auxiliary binary variable $PB[tid]$. V_i is constructed as the union of V_{i-1} and the set of all new variables $TAF[tid]$ and $PB[tid]$ for every tid in s_j . Finally, C_i is the conjunction of C_{i-1} and

$$\bigwedge_{tid \in Tid} (PB[tid] = True \leftrightarrow PP_{cvn}[tid] \wedge TAF[tid] = PB[tid] \wedge TAF[tid']),$$

where Tid is the set of all tuple ids in s_i , tid' is the id of the tuple in s_j from which the tuple tid in s_i originated in selection, and $PP_{cvn}[tid]$ is the constraint constructed from P_{cvn} by replacing every attribute name A with CVN variable $A[tid]$.

- If e_i is an aggregation $\delta[A_1, \dots, A_n, f(B)](R_j)$, $1 \leq j < i$, then Q_i is constructed as follows. s_i is constructed from $\pi[A_1, \dots, A_n](s_j)$ (with all new tuple ids), by first eliminating duplicates, and then adding the CVN(D) attribute $f(B)$, where D is the domain of values returned by aggregation f , and CVN($\{0, 1\}$) attribute TAF . V_i is the union of V_{i-1} and all new variables $f(B)[tid]$, $TAF[tid]$, for each tid in s_j , and also auxiliary variables $B'[tid]$ for every tid in s_j . C_i will be the conjunction of C_{i-1} with

$$\bigwedge_{tid \in Tid} (TAF[tid] = True \rightarrow B'[tid] = B[tid] \wedge TAF[tid] = False \rightarrow B'[tid] = 0),$$

where Tid is the set of all tuple ids in s_j , and

$$\bigwedge_{tid \in Tid} \left(f(B)[tid] = \sum_{i \in Tid(tid)} B'[i] \right),$$

where $Tid(tid)$ is the set of tuple ids in s_j that have the same values for $A_1, \dots, A_n$ as the tuple tid in s_j .

- If e_i is a constraint operation $\xi[C](R_j)$, where $1 \leq j < i$, then Q_i is constructed by taking $s_i = s_j$, $V_i = V_{i-1}$, and C_i is the conjunction of C_{i-1} and

$$\bigwedge_{tid \in Tid} (C'[tid]),$$

where Tid is the set of all tuple ids in s_j , and $C'[tid]$ is the constraint constructed from C by replacing every attribute A with the CVN variable $A[tid]$.

Finally, consider an optimization operator $\omega_{[min]}(R_j)$, where $1 \leq j < n$ and e_j is an expression that returns a table with a single value, that is, the value over the single

attribute of the single tuple (recall this must be the last operator in the sequence). It is encoded as the following MP optimization problem: Let v be a CVN variable (or a constant) corresponding to the single value in s_j . Then, the following minimization problem is being constructed as follows:

$$\min v \text{ subject to } C_{n-1}.$$

For the case of $\omega[\max](R_j)$, the reduction is the same, except that $\min$ is replaced with $\max$. Finally, if the operator is $\omega[\text{sat}](R_j)$, where sat is a key word that indicates that we are only interested in satisfiability (not optimization), the problem constructed is one that finds a feasible instantiation to variables in V_{n-1} . Note this can always be done by minimizing, or maximizing, a constant value.

Appendix B: Proof of Correctness

IN THIS SECTION, WE PRESENT A FORMAL PROOF OF correctness for both Theorem 1 and Theorem 2.

Proof 1

First, we extend the definition of *correct reduction* to *partial reduction* as follows. A partial reduction $(\{s_1, \dots, s_i\}, V_i, C_i)$ for $1 \leq i \leq n$ of a DGQL query $R_1 = e_1; \dots; R_n = e_n$; ω is correct if

- *Soundness*: Every reduction instantiation $(r_1, \dots, r_i)$ of $(s_1, \dots, s_i)$ that satisfies constraint C_i is indeed a partial feasible execution path of the DGQL query; and
- *Completeness*: Every partial feasible execution path $(r_1, \dots, r_i)$ is a reduction instantiation of $(s_1, \dots, s_i)$ that satisfies C_i .

Note that when $i = 0$, the partial reduction is $(\{\}, V_0 = \emptyset, C_0 = \text{True})$ and is defined to be correct. Also note that the correct partial reduction for $i = n$ is exactly the correct reduction.

We prove the correctness of partial constraint reduction for every $1 \leq i \leq n$ by induction on i . For $i = 0$, the reduction is correct by definition. Inductively, assuming correctness of reduction $(\{s_1, \dots, s_{i-1}\}, V_{i-1}, C_{i-1})$, we need to prove correctness for reduction $(\{s_1, \dots, s_i\}, V_i, C_i)$.

To prove soundness, let $(r_1, \dots, r_i)$ be a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$. We need to show that $(r_1, \dots, r_i)$ is a partial feasible execution path. Clearly, $(r_1, \dots, r_{i-1})$ constitutes a reduction instantiation of $(\{s_1, \dots, s_{i-1}\}, V_{i-1}, C_{i-1})$. Therefore, by induction hypothesis, $(r_1, \dots, r_{i-1})$ is a partial feasible execution path.

To prove that $(r_1, \dots, r_i)$ is a partial feasible execution path, we consider cases below corresponding to the cases of the formal reduction of $R_i = e_i$:

Case 1: e_i is a base relational table r . Since the constraints added by s_i are of the form $\text{TAF}[tid] = \text{True}$, they are all satisfied and $r_i = r$. Therefore, $(r_1, \dots, r_i)$ is a partial feasible execution path.

Case 2: e_i is a previously assigned relational name R_j , $1 \leq j < i$. Since $C_i = C_{i-1}$, $r_i = r_j$, and therefore $(r_1, \dots, r_i)$ is a partial feasible execution path.

Case 3: e_i is an *augmentation-from-domain* $\alpha[A \in D](R_j)$, $1 \leq j < i$. Since $C_i = C_{i-1}$, r_i is an augmentation of r_j , and therefore $(r_1, \dots, r_i)$ is a partial feasible execution path.

Case 4: e_i is an *augmentation-from-relation* $\alpha[A_1, \dots, A_m \in R_j](R_k)$, where $1 \leq j, k < i$. Since the constraints added to C_{i-1} to form C_i must be satisfied, the table r_i constructed as the reduction instantiation is also an augmentation of r_k . Therefore, $(r_1, \dots, r_i)$ is a partial feasible execution path.

Case 5: e_i is a projection operation $\pi_L(R_j)$, where $1 \leq j < i$. Since $C_i = C_{i-1}$, clearly reduction instantiation r_i is a projection of r_j , and therefore $(r_1, \dots, r_i)$ is a partial feasible execution path.

Case 6: e_i is an intersection operation $R_j \cap R_k$, where $1 \leq j, k < i$. The constraints added to C_{i-1} to form C_i make sure that the pairs of tuples that make TAF variable True are exactly those that are equal to each other and, therefore, must be in the intersection. Therefore, the reduction instantiation r_i is indeed the intersection of r_j and r_k , and thus $(r_1, \dots, r_i)$ is a partial feasible execution path.

Case 7: e_i is a set difference operation $R_j - R_k$, where $1 \leq j, k < i$. The constraints added to C_{i-1} to form C_i guarantee that in the instantiation of s_i , the tuples with TAF = True are exactly the tuples in r_j but not in r_k . Therefore, the reduction instantiation r_i is exactly $r_j - r_k$, and thus $(r_1, \dots, r_i)$ is a partial feasible execution path.

Case 8: e_i is a Cartesian product $R_j \times R_k$, where $1 \leq j, k < i$. The constraints added to C_{i-1} to form C_i guarantee that in the instantiation of s_i , the tuples with TAF = True are exactly those tuples composed from a pair of tuples with the corresponding TAF being True. Therefore, the reduction instantiation r_i is exactly the Cartesian product $r_j \times r_k$, and thus $(r_1, \dots, r_i)$ is a partial feasible execution path.

Case 9: e_i is selection operation $\sigma_p(R_j)$, where $1 \leq j < i$ and P is the selection predicate. The constraints added to C_{i-1} to form C_i guarantee that the tuples in the instantiation of s_i with TAF = True are exactly those that satisfy the selection condition. Therefore, the reduction instantiation $r_i = \sigma_p(r_j)$, and thus $(r_1, \dots, r_i)$ is a partial feasible execution path.

Case 10: e_i is an aggregation $\delta[A_1, \dots, A_n, f(B)](R_j)$, $1 \leq j < i$. The constraints added to C_{i-1} to form C_i guarantee that the reduction instantiation r_i is exactly the aggregation of $\delta[A_1, \dots, A_n, f(B)](r_j)$. Therefore, $(r_1, \dots, r_i)$ is a partial feasible execution path.

Case 11: e_i is a constraint operation $\xi[C](R_j)$, where $1 \leq j < i$. The constraints added to C_{i-1} to form C_i are exactly those that express C . Therefore, since C_i is satisfied, the reduction instantiation r_i satisfies C , and thus $(r_1, \dots, r_i)$ is a partial feasible execution path.

We have shown that in all cases of the formal reduction $(r_1, \dots, r_i)$ is a partial feasible execution path. Therefore, Theorem 1 is sound.

To prove completeness, let $(r_1, \dots, r_i)$ be a partial feasible execution path. We need to show that $(r_1, \dots, r_i)$ is also a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$. Clearly, $(r_1, \dots, r_{i-1})$ is a partial feasible execution path. Therefore, by the induction hypothesis, $(r_1, \dots, r_{i-1})$ is also a reduction instantiation of $(\{s_1, \dots, s_{i-1}\}, V_{i-1}, C_{i-1})$.

To prove that $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$, we consider cases below corresponding to the cases of the formal reduction $R_i = e_i$.

Case 1: e_i is a base relational table r . Since $r_i = r$, r_i is a reduction instantiation of s_i when all TAF variables are True, and thus $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$.

Case 2: e_i is a previously assigned relational name R_j , $1 \leq j < i$. Since $C_i = C_{i-1}$, $r_i = r_j$, and r_j is a reduction instantiation of s_j , r_i is a reduction instantiation of s_i . Thus, $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$.

Case 3: e_i is an *augmentation-from-domain* $\alpha[A \in D](R_j)$, $1 \leq j < i$. Since $C_i = C_{i-1}$, and r_i is an augmentation from domain of r_j , r_i is a reduction instantiation in which new variables in V_i are instantiated the with values from the augmentation. Thus, $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$.

Case 4: e_i is an *augmentation-from-relation* $\alpha[A_1, \dots, A_m \in R_j](R_k)$, where $1 \leq j, k < i$. Since r_i is an augmentation from relation of r_k , the corresponding instantiation of values into new variables in V_i must satisfy the constraints added to C_{i-1} to form C_i . Therefore, r_i is a reduction instantiation of s_i . Thus, $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$.

Case 5: e_i is a projection operation $\pi_L(R_j)$, where $1 \leq j < i$. Since $C_i = C_{i-1}$, the instantiation of values that corresponds to r_j satisfies C_i . Therefore, r_i is a reduction instantiation of s_i , and $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$.

Case 6: e_i is an intersection operation $R_j \cap R_k$, where $1 \leq j, k < i$. The constraints added to C_{i-1} to form C_i make sure that the pairs of tuples that make TAF variable True are exactly those that are equal to each other and, therefore, must be in the intersection. Since $r_i = \pi_L(r_j)$, the variable instantiation corresponding to r_i must satisfy the constraints added to C_{i-1} to form C_i , and so r_i is reduction instantiation of s_i . Thus, $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$.

Case 7: e_i is a set difference operation $R_j - R_k$, where $1 \leq j, k < i$. The constraints added to C_{i-1} to form C_i guarantee that in the instantiation of s_i , the tuples with TAF = True are exactly the tuples in r_j but not in r_k . Since $r_i = r_j - r_k$, variable instantiation corresponding to r_i must satisfy the constraints added to C_{i-1} to form C_i , and so r_i is a reduction instantiation of s_i . Thus, $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$.

Case 8: e_i is a Cartesian product $R_j \times R_k$, where $1 \leq j, k < i$. The constraints added to C_{i-1} to form C_i guarantee that in the instantiation of s_i , the tuples with $\text{TAF} = \text{True}$ are exactly those tuples composed from a pair of tuples with the corresponding TAF 's being True. Since $r_i = r_j \times r_k$, the variable instantiation corresponding to r_i must satisfy the constraints added to C_{i-1} to form C_i , and so r_i is a reduction instantiation of s_i . Thus, $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$.

Case 9: e_i is selection operation $\sigma_p(R_j)$, where $1 \leq j < i$ and P is the selection predicate. The constraints added to C_{i-1} to form C_i guarantee that the tuples in the instantiation of s_i with $\text{TAF} = \text{True}$ are exactly those that satisfy the selection condition. Since $r_i = \sigma_p(R_j)$, the variable instantiation corresponding to r_i must satisfy the constraints added to C_{i-1} to form C_i , and so r_i is a reduction instantiation of s_i . Thus, $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$.

Case 10: e_i is an aggregation $\delta[A_1, \dots, A_n, f(B)](R_j)$, $1 \leq j < i$. The constraints added to C_{i-1} to form C_i guarantee that the reduction instantiation r_i is exactly the aggregation of $\delta[A_1, \dots, A_n, f(B)](r_j)$. Since $r_i = \delta[A_1, \dots, A_n, f(B)](r_j)$, the variable instantiation corresponding to r_i must satisfy the constraints added to C_{i-1} to form C_i , and so r_i is a reduction instantiation of s_i . Thus, $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$.

Case 11: e_i is a constraint operation $\xi[C](R_j)$, where $1 \leq j < i$. The constraints added to C_{i-1} to form C_i are exactly those that express C . Since r_i must satisfy C to form a partial feasible execution path, the variable instantiation corresponding to r_i must satisfy C_i , and so r_i is a reduction instantiation of s_i . Thus, $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$.

We have shown that in all cases of the formal reduction, $(r_1, \dots, r_i)$ is a reduction instantiation of $(\{s_1, \dots, s_i\}, V_i, C_i)$. Therefore, Theorem 1 is sound.

Proof 2

Let $(\{s_1, \dots, s_n\}, V, C)$ be the reduction produced by applying Algorithm 1 to the DGQL query $R_1 = e_1; \dots; R_n = e_n; \omega$. Assume that the constraints C are feasible (otherwise Algorithm 2 returns $\perp$, which is correct). We now consider three cases:

Case 1: $\omega = \text{sat}$. The constraints C and variables V make up a constraint satisfaction problem. Because C is feasible, applying a CSP solver will produce an assignment of variables in V that satisfies C . We then apply the steps in Definition 6 to get the reduction instantiation $(r_1, \dots, r_n)$. By Theorem 1 every reduction instantiation is a feasible execution path; thus $(r_1, \dots, r_n)$ is correct.

Case 2: $\omega = \min(R_k)$. The expression R_k returns a single numeric value that is held in the variable V_{R_k} . This is the objective in the mathematical programming problem: $\min V_{R_k}$ subject to C . Because C is feasible, applying an MP solver

will produce an assignment of variables in V that satisfies C such that V_{R_k} is minimum. We then apply the steps in Definition 6 to get the reduction instantiation $(r_1, \dots, r_n)$. By Theorem 1 every reduction instantiation is a feasible execution path. Assume that r_j is not the minimal value over all feasible execution paths. Then the reduction instantiation is not minimal, which contradicts the fact that the MP solver found the minimal. Thus $(r_1, \dots, r_n)$ is correct.

Case 3: $\omega = \max(R_k)$. This follows from the proof of Case 2 and the observation that $\max V_{R_k}$ subject to C is equivalent to $\min - V_{R_k}$ subject to C .

